

 他要冒蓝火儿

jwt与token+redis，哪种方案更好用？

11492 浏览 | 0 回复 | 2020-08-27



他要冒蓝火儿

[+关注](#)

在设计no session系统时，遇到了有两种可选方案：jwt与token+redis。

JWT: 生成并发给客户端之后，后台是不用存储，客户端访问时会验证其签名、过期时间等再取出里面的信息（如username），再使用该信息直接查询用户信息完成登录验证。jwt自带签名、过期等校验，后台不用存储，缺陷是一旦下发，服务后台无法拒绝携带该jwt的请求（如踢除用户）；

token+redis: 是自己生成个32位的key，value为用户信息，访问时判断redis里是否有该token，如果有，则加载该用户信息完成登录。服务需要存储下发的每个token及对应的value，维持其过期时间，好处是随时可以删除某个token，阻断该token继续使用

Web认证是任何一个认真一点的网站都必须实现的基本功能。这个功能解决了让服务器“认识你就是你”的问题。这个功能看起来貌似很简单，但是实际上处处是坑。因为认证是依靠一套技术整体运作才能完成，所以仅仅是把一些现成的技术简单拼起来是不够的。你必须了解每一种技术能做什么，不能做什么，解决了哪些问题，才能精心设计一套认证功能。

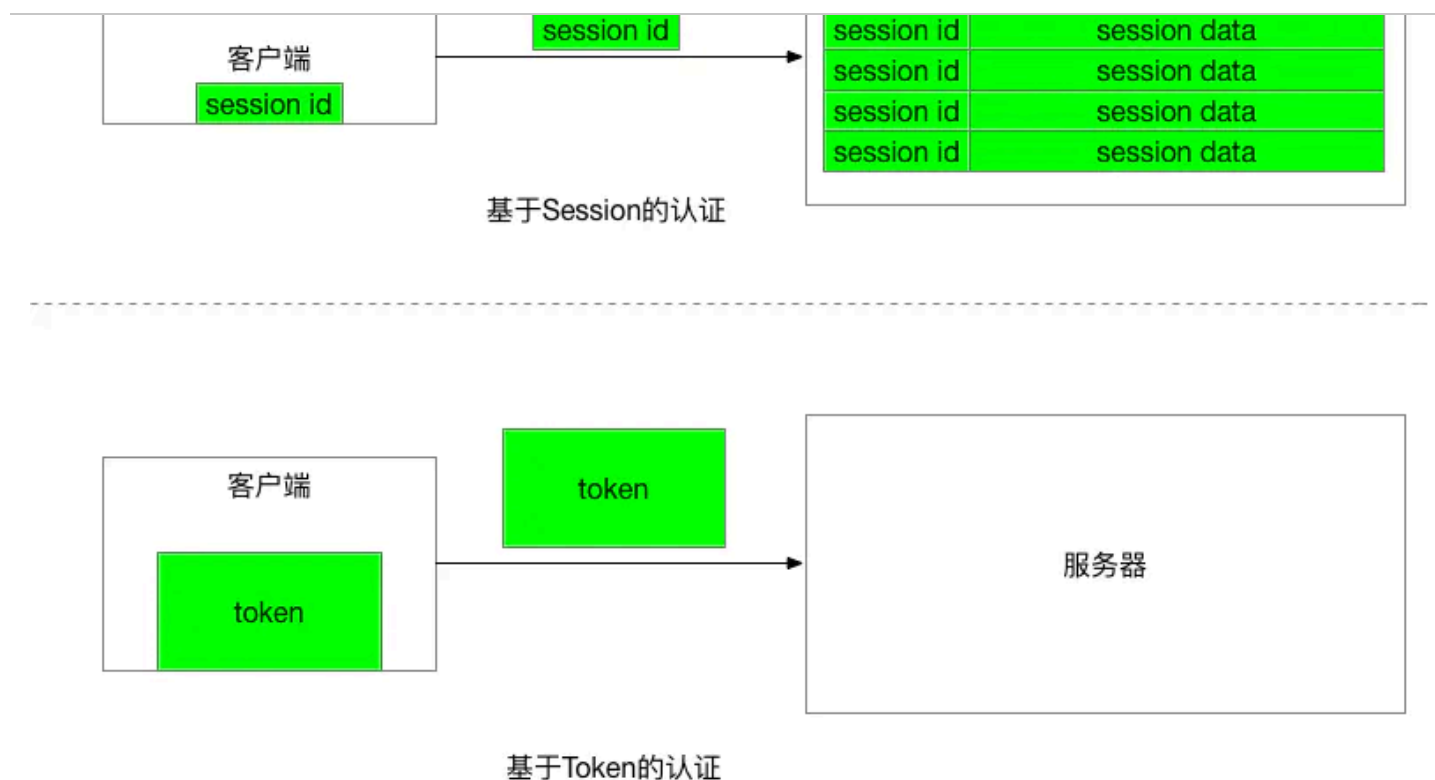
两种认证

目前市面上能见到的认证方式分为两大种——基于Session的和基于Token的。

所谓**基于Session的认证**，是指在客户端存储一个Session Id。认证时，请求携带Session Id，并由服务器从Session数据存储中找到对应的Session。这种方式在很多网站框架下都有

所谓**基于Token的认证**，是指将所有认证相关的信息在服务器端编码成一个Token，并由服务器签名，以确保不被篡改。Token本身是**明文**的。存在Token里的信息可以有比如user id、权限列表、用户昵称一类的。这样服务器只要拿着token和token的签名，就可以直接验证用户的身份是合法的。在现实当中，基于Token的认证的主要标准是Json Web Token（JWT），见[RFC 7519](#)。

☰ 他要冒蓝火儿



认证方式

但是我不得不说的是，基于Token的认证在现实当中并不是很实用……

JWT

一个JWT大概长这样：

```
1 | base64(header).base64(json payload).signature
```

- header部分描述一些基本信息，比如这个token是用什么算法签名的，是什么版本的等等。
- payload就是一个json object。你可以任意放置你想要的信息，只要符合json的格式即可。标准中已经规定好了有一些字段的意思，比如iat表示issue at，token签发的时间；exp表示token过期的时间等等。根据这些约定就可以实现一些小的代码库来检查比如token是不是过期了等等。但是请注意，很多人误解，认为JWT是加了密的，但其实**payload是明文的**。
- signature是一个签名。服务器端可以自行选择一个算法和一个secret，与payload拼接上，得到一个签名。secret并不会在网络中传输，所以客户端无法伪造一个JWT。这样，一旦一个签名生成，再传回给服务器，服务器就可以知道这个token是不是它当初生成的。

通过这样的机制，JWT中可以存储一些认证必要的信息。给定一个JWT，服务器只要验证：

- 这个JWT的签名是对的
- 这个JWT还在生效（即当前时间在JWT生效时刻之后，在失效时刻之前）

之后服务器就可以信任这个JWT中包含的信息，包括user id、包含的权限等等。服务器不需要自己再去查询一遍这个用户的信息，以及这个用户的权限信息，就可以对请求作出相应。不用session了，无

☰ 他要冒蓝火儿

人登录了多少次，登陆了什么设备；同时，也无法强行“踢”掉一个用户的登录——JWT一旦生成，在失效之前，总是有效的。如果实现了一个token黑名单之类的功能，就等价于实现了Session机制，无状态带来的好处就无从谈起。这个限制对于任何一个要认真做用户风险控制的网站来说都是不可能接受的。

- 使用了JWT，无法很好的控制payload的数据量。尽管规范表示，应该只把认证的相关信息放到payload里。但实际上，开发人员往往会误用，把几乎所有和user相关的数据都放到payload里。而payload的尺寸过大，比如达到数KB，就会极大的损耗带宽和IO性能。要记得，为了达成“无状态”，每个请求都必须把全量的JWT都带着……

这两个严重的缺陷限定了JWT只能用到一些不太认真的场景。而对于真正的社交、金融、游戏等认真一点的服务，还是要选择基于Session的认证。

当然，token中的签名还是有好处的，签名可以确保token的确是服务器产生的，不会被篡改。如果token中包含了user id，那么还可以实现简单的前端错误上报；如果token中还有session id，就可以在服务器端实现基于Session的认证。因此，你可以将user id、session id、token过期时间等几个关键数据放到payload里——只放这几个，不放其他的数据，得到一个用来做Session认证的JWT。更进一步，如果你把JWT的规范稍微小改一下，比如payload不用JSON，而是更紧凑的格式；定死了签名算法，即可省略JWT的header了；最后再优化一下编码格式，就能得到一个你自己的token。

但，无论用session还是token，还是什么其他的名字，这些都不重要。重要的是服务器这边**必须实现session机制**，以便于对用户登录信息进行有效的管理。

有人告诉过我一个使用基于Token + 无状态的认证方式的原因：他们的存储是一个云服务，并且按照调用次数收费。所以他们让用户每次将Token传给服务器，就是希望尽量少的调用那个云服务。对此我表示很无语……

怎么存储认证信息

谈完了session和token，我们来说这个信息在客户端怎么存储。客户端也算分两大类——浏览器和Native App。先说说浏览器。

浏览器

浏览器中的存储主要是Local Storage和Cookie。

其实浏览器用于存放认证信息的存储还有Session Storage，但是它和Local Storage差不多，只是失效的机制不太一样。这里只用Local Storage讨论。

使用基于Token认证的开发人员很喜欢使用Header + Local Storage。因为这样可以有效防止CSRF（下一小节专门讲）。

☰ 他要冒蓝火儿

替用户登录了。

整个浏览器中, 只有一种资源是脚本无法访问到的。这就是**被设置为HttpOnly的cookie**。这是非常理想的放置认证token/session id的地方。设置这种token只需要在Set Cookie时这么写:

```
1 | Set-Cookie: access_token=xxxxxxxxxxxxxxxxxxx; HttpOnly; Secure; Same-Site=strict; Path=/;
```

(Secure和Same-Site是什么? 下文会解释)

XSS攻击者没有任何办法从HttpOnly的Cookie中拿到你的认证信息, 除非他能在你登录网站后, 直接进入你的电脑, 打开浏览器的开发者工具并人肉复制粘贴(叫你不锁屏, 哼)。

有些人坚称自己的程序可以保证不受XSS的攻击, 所以可以放心的用Local Storage。比如使用React框架开发的程序理论上所有的DOM节点都由React的虚拟DOM产生, 所有的标签生成都进行了escape。escape掉的脚本就无法执行, 也就不可能XSS了。

这样讲没有错误, 但是XSS最令人头疼的地方在于你很难保证你的网站对所有用户的输入都进行了escape。

- 你编写的是一个写文章的网站, 需要支持用户手工输入HTML, 并且HTML必须得直接展示;
- 你编写的网站99%是React这样的框架生成的, 但是可能会有一些边角, 为了方便还是使用jquery等传统技术
- 你的网站是一个团队开发, 尽管开发规范要求大家都要对用户的输入进行escape处理, 但是只要是人就会忘, 而escape这件事情不一定能进入到测试的Case清单;
-

只要有一个漏洞存在, 那么整个防护体系就完全失效。这就是为什么HttpOnly Cookie这样的绝对隔离措施很关键的原因。

Native App

Native App比浏览器相对简单。一般Native App都是静态编译产生, 不存在XSS的问题。同时移动操作系统都会有沙箱机制, 避免其他App读取到自己的数据(除非你root了……)。因此, Native App可以比较放心的将数据存在App沙箱内某个存储即可。如果不放心, 可以考虑如iOS KeyChain或者Android KeyStore这样的设施。

但Native App与服务器交互有一些区别。Native App一般是不直接支持Cookie机制的。所以如果一个服务器端使用Cookie来保存认证信息, 就需要Native App手工解析Set-Cookie Header, 同时, Native App因为不直接支持Cookie, 所以倾向于在请求中使用AuthorizationHeader来传入认证信息。这也需要服务器适配。当然, 最简单的办法是让Native App引入一个模拟Cookie行为的库。

防止CSRF

CSRF代表**Crossing Site Resource Forge**。大致的触发流程是:

☰ 他要冒蓝火儿

户去点击。当用户点击时，会发出到A站点的请求。而浏览器会给这个请求附上A站点的认证信息，从而让这个请求能够执行。这种行为可能是，但不限于，给某个A站点的某个其他用户提权/转账/发文辱骂等等。

上文中提到了，很多人用JWT+Local Storage的本心是为了防护CSRF。这样做的原因是——因为Cookie的发送是完全由浏览器控制的，不受网页本身的控制。所以最简单直接的办法，就是不用Cookie，不让自动发送认证信息成为可能。问题在于，**这么干是有XSS风险的**。从上文中可以看到，为了避免XSS，就必须用HttpOnlyCookie。

那么怎么在使用Cookie的同时，还能防范CSRF呢？

传统页面Web网站

在传统页面Web网站中，一般会使用**CSRF Token**。这是个非常流行的做法。像Tomcat这类的容器都会自带CSRF Token的产生和检查Filter。

CSRF Token是这样工作的。客户端要首先向服务器请求一个带有提交表单的页面，服务器返回的页面中会嵌入一个CSRF Token。当用户提交表单时，CSRF Token会被一起携带发给服务器做验证。所以当服务器看到CSRF Token，就可以放心大胆的确认**用户的的确确是看到了提交前的表单界面**，从而避免了用户稀里糊涂提交一个被伪造的表单的可能性。

SPA

CSRF Token只适合于传统的页面请求，在SPA的情况下会比较尴尬。

在SPA中，客户端与服务器之间的交互主要是通过接口完成的，没有页面的概念。此时，你的确可以照猫画虎的做一个接口让用户拿到CSRF Token，但这样**什么也确认不了**。因为攻击者可以调用同样的接口，拿到合法的CSRF Token。

这时有几种办法：

- **给所有接口都添加一个请求secret**，来标记其来自于合法的客户端。这个secret可以是固定的随机字符串，也可以通过某些动态算法产生。对于CSRF，浏览器只会做自动传Cookie而已，并不能帮助传入secret。这样一来，就可以确定消除CSRF的风险。但注意，这个机制仅能防范CSRF，而不能防范人为的攻击。黑客只要拿得到客户端，就一定能找到生成secret的办法。

secret有一个顺带的功能是提高了第三方用户随意调用接口的门槛——他们必须得去查看客户端源代码，学会了怎么生成secret才能调用接口。

- **用Same-Site Cookie**。回到上面CSRF步骤的第二步骤。当用户看到了B站点伪造的表单，点击了提交，向站点A发出请求时，被标记了Same-Site=strict的Cookie是不会被携带的，因为当时的主站点域名B和提交的站点域名A不一样。这是Same-Site=strict标记是个相对较新的标准。目前大部分浏览器都已经支持了。但如果大量的用户群还在类似于IE8这样的老系统上，这个招数便是无效的。

☰ 他要冒蓝火儿

允许跨域的，所以天然的屏蔽掉了这个问题。传统表单的提交的格式必然是application/x-www-form-urlencoded。因此只要保证服务器能够拒绝处理所有application/x-www-form-urlencoded格式的POST请求，就能确保SPA不受CSRF的影响。那用啥呢？JSON - application/json。（我专门写这一条的原因是，jquery的ajax库的默认行为正是使用application/x-www-form-urlencoded格式。如果你还在用，可以考虑改一下。）

- **另一个歪招，双认证。**将你的认证信息同时放在HttpOnly Cookie和Authorization Header。服务器要先比对这两个值是一样的，然后再去执行认证过程。这样可以同时防范XSS和CSRF。代价是，如果你的认证信息比较长，会浪费一些带宽。

总是使用https

大学上网络课时，老师讲解了http的一些原理，然后给我们留了个作业——去外边提供WIFI的餐厅用嗅探器扒别人的密码。两周后，我们做完了作业，心情是悲催的——尼玛互联网都发明了十几年了，连最基本的防护都没有……

http是明文传输的。在http下，用户输入的任何信息，从他的电脑到服务器之间的每个链路节点都是明文的。在这里个链路中的任何地方，都可以截取到完整的数据，包含你的密码，认证token……

这就是为什么https是**必须的**。https主要提供三个保证：

- 端端加密。通过https交互的原始数据，只有用户的浏览器和最终的服务器可以看到。其他中间节点无法获）。
- 客户端可以认定**要访问的服务器就是那个服务器**。这是被证书体系所支撑的。一旦浏览器的地址栏出现了网址的证书信息，并且是绿色的提示，那么用户的心里就可以稳了。（当然国内其实也不完全是这样，讲多了查水表，懂者自懂）。
- 服务器可以认定**访问的客户端就是合法的客户端**。这种模式被称为“2-Way SSL”或者“Mutual SSL”。这种模式是可选的，需要多配置一个客户端证书，一般场景用不到，多见于企业Web服务。

早些时候，很多人对https有一些抵触，大致的原因是，支持https需要软件改造；服务器对证书进行密码***算要耗费很多CPU；同时也会带来跟多的网络请求和响应（多了ssl握手）。这无疑会带来一些成本和体验上的问题。但那已经是10多年前的事情了。现在的软硬件处理能力和网络基础设施比起10年前都有数倍的提高。如果今天，一个商业网站仍然坚持不用https，那么可以请他的老板去大街上裸奔。

使用了https后，为了进一步保证安全，将Cookie设置为Secure。这样，浏览器就可以只在访问https网址时才会携带Cookie。如果不做这样的设置，通过https站点设置的Cookie，仍然会向http站点发送。当这个站点的域名解析被劫持，就可能造成向一个伪造的服务器发出你的认证信息。

认证信息不应该永久有效

很多人为了“用户体验”，选择让一个登录永久有效。这样做是非常危险的。因为一旦用户的认证信息被别人获取了，就永久性的失去了防御的机会（记得上面说的不锁电脑屏幕的后果吗？）。

⋮ 他要冒蓝火儿

但，为了让频繁使用的用户体验更好，可以考虑实现**会话期续期**。但需要注意，这里说的续期是指从用户角度看可以延续其不需要登录的时间长度，而不是直接让session/token有效期变长。必须实现为**给用户替换一个新的session id/token**。这样做，既能保证同一个认证信息不会永久有效，又能让正常的、频繁使用的用户免除登录之苦。

总结一下

总结下来，一个靠谱的Web认证应该：

- 可以使用Session也可以使用Token做认证，但是总是要保证服务器端可以管理Session，通过Session是否存在来最终确定认证的有效性；
- 将认证信息存放在标记为HttpOnly, Secure, Same-Site=strict的Cookie中，从而避免XSS和CSRF；
- 总是使用https，只要你的网络链路经过了公网；
- 如果是传统的页面网站，请使用CSRF Token机制；
- 如果可以，做一个简单的请求secret，可以辅助防止CSRF，也可以稍稍的提高接口被爬取的门槛；
- 如果是SPA应用，放心大胆的禁用对application/x-www-form-urlencoded的支持
- 保证token/session必须有一个有效期

如果你也觉得靠谱，就不妨照着做。

作者：大宽宽

链接：<https://www.jianshu.com/p/805dc2a0f49e>

来源：简书

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

举报



收藏 5



赞 1

0条评论

🔼 默认排序 ▾



☰ 他要冒蓝火儿

请留下你的观点吧~

发布

 牛客博客，记录你的成长

[关于博客](#) | [意见反馈](#) | [免责声明](#) | [牛客网首页](#)